



SIZING RUNBOOK

VIRTUAL CONTROL

VMWARE CLOUD FOUNDATION SOLUTIONS

Fleet Appliance Sizing Increase Runbook

Why VCF 9 Fleet ships undersized for active deploys, what fails when it stays at default (4 vCPU / 12 GB), the verified vCPU/RAM resize procedure, and the host reservation ceiling that can silently block memory increases.

FLEET

VCPU

HOT-ADD

VCFA BOOTSTRAP

LEADER ELECTION

VCF 9.0.1

VMware Cloud Foundation

Fleet Appliance Sizing Increase Runbook

Prepared by: Virtual Control LLC **Date:** May 18, 2026 **Document Version:** 1.0 **Classification:** Internal — Lab Environment **VCF Version:** 9.0.1.0 **Cluster:** vcenter-cl01 — 4-host nested ESXi on Dell Precision 7920

Scope. This runbook documents the verified procedure to grow Fleet from its default 4 vCPU / 12 GB to a deploy-capable sizing (6 vCPU minimum, 8 vCPU / 16 GB recommended). It covers the diagnostic signature that tells you Fleet is undersized (KIND control plane crash loop), the Broadcom KB reference (KB 426412), the hot-add-disabled-by-default fact (power cycle required), and the **memory-edit ceiling** that can silently reject a memory increase when host-level VM reservations consume the headroom.

Companion docs. - [Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md](#) — host RAM rebalance procedure that may affect the memory-edit ceiling - [Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md](#) — cleanup procedure for failed envs - [Nested-vSAN-WilliamLam-Tunings-Runbook-2026-05-19.md](#) — bootstrap-VM-side storage tunings (separate layer)

Table of Contents

1. TL;DR
2. Symptom and Trigger
3. Diagnostic Signature
4. The Root Cause
5. Decision: Target Sizing
6. The Procedure (Verified)
7. Memory-Edit Ceiling (Why a RAM Increase May Be Silently Rejected)
8. Post-Resize Verification
9. Operating Principles
10. Document Information

1. TL;DR

VCF 9 Fleet Management appliance ships with **4 vCPU / 12 GB RAM** by default. That sizing is **insufficient** for any deploy that exercises Fleet's internal KIND Kubernetes cluster (VCFA, VCF Operations for Networks, etc.). Under load, the KIND control plane (etcd, kube-apiserver, kube-controller-manager, kube-scheduler) hits leader-election timeouts and crash-loops, causing the bootstrap workflow to fail with `LCMVSPHERECONFIG1000095` after hours of wall time.

Recommended target: 8 vCPU / 16 GB minimum. 6 vCPU / 12 GB is a verified-sufficient floor that prevents the crash loop when a memory increase is blocked by host reservation ceilings — see Section 7.

Procedure summary: 1. Snapshot Fleet 2. Graceful guest shutdown 3. vSphere → Edit Settings → CPU = new target 4. Power on, wait 15–20 min per KB 426412 for services to initialize 5. Verify

Hot-add is disabled by default on Fleet — power cycle is required for resize. Memory hot-add is also disabled.

2. Symptom and Trigger

2.1 When this runbook applies

Apply this runbook when a VCFA (or any deploy that exercises Fleet's internal KIND cluster) stalls and Fleet's KIND control plane shows restart counts climbing.

2.2 Diagnostic symptoms

- Deploy submitted via API, accepted (HTTP 200), env created in Fleet DB
- After 1-2 hours, env stays `INPROGRESS` but no visible progress in bootstrap log
- Bootstrap VM `vcfa-mgmt-*` may or may not be created (depends on which controller is alive at submission time)
- Fleet load average: sustained $>3\times$ nproc (e.g. 8+ on 4 vCPU)
- Fleet memory: typically 50–60% free — **memory is not the bottleneck**
- `docker exec kind-bootstrap-control-plane kubectl get pod -n kube-system -l 'tier=control-plane'` shows:
- `kube-controller-manager-kind-bootstrap-control-plane` : **restart count climbing every ~2 min**
- `kube-scheduler-kind-bootstrap-control-plane` : **restart count climbing every ~2 min**

This is the **single most diagnostic indicator** — controller-manager and scheduler restarting every ~2 min, while etcd and apiserver appear stable.

3. Diagnostic Signature

3.1 The crash pattern in pod logs

```
E<date> <time> leaderelection.go:332]
  error retrieving resource lock kube-system/kube-controller-manager:
  Get "https://172.18.0.2:6443/apis/coordination.k8s.io/v1/namespaces/kube-system/leases/kube-controller-
  manager?timeout=5s":
  net/http: request canceled (Client.Timeout exceeded while awaiting headers)

E<date> <time> leaderelection.go:332]
  ... context deadline exceeded

I<date> <time> leaderelection.go:285]
  failed to renew lease kube-system/kube-controller-manager:
  timed out waiting for the condition

E<date> <time> controllermanager.go:302]
  "leaderelection lost"
```

3.2 What the pattern means

- controller-manager and scheduler each renew their leader-election lease every few seconds via the kube-apiserver
- Lease-renewal call has a **5-second client timeout** baked into the K8s libraries
- Under CPU saturation, kube-apiserver responds in >5s — renewal request is cancelled
- The pod logs `leaderelection lost` and exits
- kubelet restarts it
- It re-acquires the lease successfully (apiserver is intermittently responsive)
- Cycle repeats: crash, restart, re-elect, crash — about one cycle every 2 min
- **No K8s reconciliation can complete stably** — the BootstrapVMSPTask workflow can never progress

3.3 Verifying CPU saturation as the cause (not RAM, not disk)

```
# On Fleet appliance via SSH:
nproc # current vCPU count
uptime # load avg – compare to nproc
free -h # confirm memory has headroom (not bottleneck)
iostat -x 1 2 # confirm disk wait/util low (not bottleneck)
top -bn1 -o %CPU # confirm apiserver + etcd consume most CPU
```

When `load avg / nproc > 1.0` sustained, you are CPU-bound. A 4 vCPU Fleet idling at load 5.8 is already 145% saturated, and rises to 200%+ once a deploy is in flight.

4. The Root Cause

Fleet's default 4 vCPU sizing is sufficient for steady-state Fleet operations (Java vrlcm-server polling, REST API serving) but is insufficient for active deploy workloads. When a VCFA deploy is in flight, Fleet runs:

1. Its own Java services (vrlcm-server, lookup_passwords, nginx, vpostgres) — baseline ~2 vCPUs
2. A KIND inner-Kubernetes cluster (etcd, kube-apiserver, kube-controller-manager, kube-scheduler, coredns, kindnet, kube-proxy) — baseline ~1 vCPU
3. CAPI/CAPV controllers (cluster-api-provider-vsphere, cert-manager, trust-manager) — baseline ~0.5 vCPU
4. The actual BootstrapVMSPTask workflow that ssh/govc-talks to the new bootstrap VM — baseline ~0.5 vCPU

That's ~4 vCPUs at idle. Any spike (image pull, OCI push, K8s reconciliation burst) immediately saturates the 4-vCPU host, and the 5-second leader-election timeout in the K8s control plane is the most brittle thing — it crashes first.

Memory is typically NOT the bottleneck. Fleet usually has 50–60% RAM free even at full deploy load. The fix is vCPU, not RAM — though many environments will also benefit from more memory for the etcd page cache.

5. Decision: Target Sizing

5.1 Broadcom-recommended sizing

VCF 9 release notes and the Fleet Management techdocs page do not publish an explicit minimum sizing increase for active deploys. The default OVA-shipped 4 vCPU / 12 GB is documented as the "small" Fleet profile.

5.2 Verified sizing breakpoints

vCPU	Sustained idle load	Deploy spike load	Outcome
4	1.2–2.0 (saturated)	8+ (4x saturation)	Crash-loop
6	0.4–0.7 (healthy)	2–3 (manageable spike)	Verified stable
8 (recommended)	0.3 (relaxed)	1–2 (comfortable)	Best

For VCFA deploys specifically: **6 vCPU is the verified minimum that prevents the crash-loop. 8 vCPU is the recommended target.**

5.3 Memory target

RAM	Steady state	Heavy deploy	Outcome
12 GB (default)	4–5 GB used	8–9 GB used	Functional but no page-cache headroom for etcd
16 GB (recommended)	4–5 GB used	8–9 GB used	Comfortable; ample page cache

Memory is rarely the immediate cause of the crash-loop, but it is recommended bumped at the same time as vCPU for future deploy headroom. See Section 7 for the host reservation ceiling that may prevent the memory increase from being accepted.

6. The Procedure (Verified)

6.1 Pre-flight

```
# Verify Fleet's current sizing
$fleet = Get-VM 'fleet'
$fleet | Select-Object Name,PowerState,NumCpu,@{N='Mem_GB';E={$_.MemoryGB}},
                    @{N='Host';E={$_.VMHost.Name}},
                    @{N='CpuHotAdd';E={$_.ExtensionData.Config.CpuHotAddEnabled}},
                    @{N='MemHotAdd';E={$_.ExtensionData.Config.MemoryHotAddEnabled}}
```

Expected on a default Fleet: - NumCpu = 4 - MemoryGB = 12 - CpuHotAddEnabled = **False** - MemoryHotAddEnabled = **False**

Hot-add is disabled by default. Power cycle is required.

6.2 Snapshot

```
New-Snapshot -VM (Get-VM 'fleet') `
    -Name "pre-vcpu-resize-$(Get-Date -Format yyyy-MM-dd)" `
    -Description "Before increasing Fleet vCPU/RAM to fix K8s control plane crash loop" `
    -Memory:$false -Quiesce:$false
```

6.3 Graceful guest shutdown

```
Shutdown-VMGuest -VM (Get-VM 'fleet') -Confirm:$false
# Poll until PoweredOff (typically 30-60 sec)
while ((Get-VM 'fleet').PowerState -ne 'PoweredOff') {
    Start-Sleep -Seconds 5
}
```

6.4 Reconfigure (Edit Settings)

Either via vSphere UI: **Edit Settings** → **CPU = 8 (or 6)** → **Memory = 16384 MB** → **OK**, or via PowerCLI:

```
Set-VM -VM (Get-VM 'fleet') -NumCpu 8 -MemoryGB 16 -Confirm:$false
```

If memory edit fails, see Section 7.

6.5 Power on

```
Start-VM -VM (Get-VM 'fleet') -Confirm:$false
```

6.6 Wait per KB 426412

[KB 426412 — VCF Operations "no healthy upstream"](#) prescribes a 15–20 min wait after any Fleet power-on before checking service health. Fleet's services initialize at different times and the appliance is not considered "ready" before this window passes.

Do not poke at services or the UI before 15 min. Load average will spike during boot (10–20 sustained) and then drop sharply when init completes.

7. Memory-Edit Ceiling (Why a RAM Increase May Be Silently Rejected)

Operational gotcha: the vCPU edit may succeed while the memory edit is **silently rejected** by the vSphere UI. The reservations on other VMs on the target host can prevent a memory increase even when raw "free" looks adequate.

7.1 The symptom

You click **Edit Settings** → **Memory = 16 GB** → **OK** and the dialog either closes without applying the change or returns an admission-control error. Raw host free memory looks sufficient (e.g. 32 GB free on a 64 GB host being asked for a +4 GB delta), but the change is refused.

7.2 Why it happens

vSphere's reconfigure admission check sums **VM-level reservations** of every VM on the host, not just live usage. When other management VMs (vCenter, NSX, VCF Ops, etc.) have explicit memory reservations, those reservations consume the headroom even if those VMs are not using the full reserved amount. Combined with the implicit per-

VM memory overhead (~6% of allocated), the total reservation + overhead can exceed the host's physical RAM once Fleet's allocation grows.

7.3 How to recover

In order of preference:

1. **vMotion Fleet off the constrained host** to one with more reservation headroom, then retry the memory edit.
2. **Lower the reservations on the OTHER VMs on the host** — only if you know why those reservations exist; many appliance OVAs ship with hard reservations for a reason.
3. **Add more physical RAM to the host** (impractical mid-deploy).
4. **Accept the vCPU-only increase.** 6 vCPU / 12 GB is verified sufficient to stabilize the K8s control plane even without the RAM bump, because the crash-loop is CPU-bound not memory-bound.

8. Post-Resize Verification

After power-on + 15–20 min wait, run this on Fleet via SSH:

```
echo "=== nproc shows new CPU count ==="
nproc

echo ""
echo "=== Load average and uptime ==="
uptime
# load avg / nproc should be < 1.0 for steady state

echo ""
echo "=== Critical services ==="
systemctl is-active docker vrlcm-server vmware-vpostgres lighttpd nginx

echo ""
echo "=== Failed units ==="
systemctl --failed --no-pager

echo ""
echo "=== If KIND was up before – verify control plane stable ==="
# (Note: vrlcm-server only spins up KIND for active deploys;
# if no env is in flight there will be no kind-bootstrap container,
# which is the expected idle state)
docker ps --format "table {{.Names}}\t{{.Status}}"

echo ""
echo "=== Inbound REST works ==="
curl -sk -o /dev/null -w "HTTP %{http_code}\n" \
  https://localhost/lcm/lcops/api/v2/environments \
  -u 'admin@local:<password>'
```

Pass criteria: - `nproc` reflects the new vCPU count - Load avg / nproc < 1.0 (healthy headroom) - All 5 systemd services `active` - No failed units - HTTP 200 from REST

If any check fails, do NOT immediately roll back. Wait an additional 10–15 min — some services have late initialization. If still failing after 30 min total, then revert from the pre-resize snapshot.

9. Operating Principles

1. The default 4 vCPU / 12 GB Fleet sizing is right-sized for idle Fleet operation only. Any active deploy workload (VCFA, VCF Operations for Networks, etc.) will push Fleet past CPU saturation. Plan for 8 vCPU / 16 GB as the working minimum for any deploy-capable Fleet instance.

2. Hot-add is disabled on Fleet by default. Both `CpuHotAddEnabled` and `MemoryHotAddEnabled` are `False`. Power cycle is required for resize. This is contrary to many appliance norms (vCenter has hot-add on by default; Fleet does not).

3. The crash-loop signature is unambiguous and easy to diagnose remotely. `kubectl get pod -n kube-system -l 'tier=control-plane' -o custom-columns=NAME,READY,RESTARTS,AGE` taken twice 2 min apart will show if controller-manager/scheduler restart counts are increasing. Increasing counts + 5-second leader-election timeouts in pod logs = CPU starvation.

4. KB 426412 prescribes a 15–20 min wait after Fleet power-on. Do not skip this. Pre-warmup interventions cause more problems than they solve.

5. Memory edits can be silently rejected by host reservation ceilings. Especially after a recent host RAM rebalance or evacuation, the host's reservation accounting may forbid a delta that "should fit" by free-space math. If a memory edit fails: try vMotioning Fleet to a less constrained host first.

6. vCPU is almost always the actual cause of the crash-loop, not memory. Memory is typically 50–60% free throughout. Bumping memory to 16 GB is good practice but not strictly necessary for the leader-election timeout symptom.

7. Fleet sizing fixes a Fleet-side bottleneck only. Bootstrap-VM-side bottlenecks (etcd latency on nested vSAN, etc.) are separate layers with their own runbooks. Treat sizing as additive, not exclusive.

10. Document Information

Field	Value
Title	Fleet Appliance Sizing Increase Runbook
Document ID	VC-RUNBOOK-FLEET-SIZING-20260518
Version	1.0
Issued	May 18, 2026
Author	Virtual Control LLC
Classification	Internal — Lab Environment
Related Documents	Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md , Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md , Nested-vSAN-WilliamLam-Tunings-Runbook-2026-05-19.md , VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md
Distribution	Lab notebook

Revision History

Version	Date	Notes
1.0	May 18, 2026	Initial release. Verified procedure to grow Fleet from default 4 vCPU / 12 GB to a deploy-capable sizing. Covers the diagnostic crash-loop signature (controller-manager + scheduler restart counts climbing due to 5-second leader-election timeouts on saturated CPU), verified sizing breakpoints (4=crash, 6=stable floor, 8=recommended), the resize procedure (snapshot → graceful shutdown → reconfigure → power on → 15–20 min wait per KB 426412), the hot-add-disabled-by-default constraint, and the host reservation ceiling that can silently reject a memory increase.

Document End

See also: - [Nested-ESXi-RAM-Rebalance-for-VCFA-Deploy-2026-05-18.md](#) — host RAM rebalance procedure that may affect the memory-edit ceiling - [Failed-VCFA-Deployment-Cleanup-Runbook-2026-05-18.md](#) — cleanup procedure for failed envs - [Nested-vSAN-WilliamLam-Tunings-Runbook-2026-05-19.md](#) — bootstrap-VM-side storage tunings (separate layer) - [VCFA-Simple-Deploy-Wizard-Precheck-Gotchas-2026-05-18.md](#) — wizard-level gotchas to verify before submit